

Национальный исследовательский университет ИТМО
Факультет программной инженерии и компьютерной техники

Отчёт к практическому заданию №3
по дисциплине «Системное программное обеспечение 2»

Студент: Горинов Даниил Андреевич
Группа: Р4116

Санкт-Петербург
2025/2026 учебный год

1 Цели

Реализовать поддержку операций извлечения данных для структуры данных по варианту с использованием комплекса программ первого семестра.

Для варианта 4 структура данных — файловая система Btrfs. Программа должна определить поддержку файловой системы, перейти в диалоговый режим PASSIVE FTP и поддерживать просмотр директорий, переход по каталогам и копирование файлов или директорий из образа.

2 Задачи

1. Изучить on-disk формат Btrfs и способы инспектирования образа.
2. Подготовить настоящий Btrfs-образ с тестовым деревом файлов.
3. Сформировать таблицы inode, dirent и extent для VM-программы.
4. Подключить образ к VM через BlockDevice.
5. Реализовать FTP-команды PWD, CWD, LIST, NLST, SIZE, RETR, COPY и служебные команды клиента.
6. Проверить положительный сценарий, отрицательный сценарий с неподдержанной ФС и подключение обычного FTP-клиента.

3 Описание работы

Образ создаётся скриптом `tools/prepare_btrfs_image.sh`. На Linux он использует локальные `btrfs-progs`; на macOS запускает контейнер Alpine с `mkfs.btrfs` и `mount`. В образ копируется дерево SP08, исключая генерируемые результаты, бинарники, `.git` и сам образ.

Схема подготовки:

```
dd if=/dev/zero of=spo8_btrfs_block.img bs=1M count=160
mkfs.btrfs -f -L SP08_REAL spo8_btrfs_block.img
mount -o loop spo8_btrfs_block.img /mnt/spo8
rsync ./SP08/ /mnt/spo8/SP08/
umount /mnt/spo8
```

После размонтирования сохраняются дампы:

- `results/btrfs_real_super.txt`;
- `results/btrfs_real_chunk.txt`;
- `results/btrfs_real_tree.txt`;
- `results/btrfs_source_files.txt`.

По этим дампам `tools/gen_btrfs_ftp_asm.py` генерирует таблицы для `results/btrfs_ftp_desc`. Байты файлов в ASM не вшиваются: при RETR и COPY VM читает данные из настоящего `spo8_btrfs_block.img` через BlockDevice.

Запуск основного сценария:

```
make -C SP08 remote-demo
```

Проверка FTP-клиента:

```
make -C SP08 remote-ftp-smoke
```

Для ручной проверки используется FileZilla с параметрами 127.0.0.1:2121, обычный FTP без TLS, пассивный режим, анонимный вход.

4 Аспекты реализации

В `spo8.target.pds1` добавлены инструкции для взаимодействия с устройствами:

- `PIPE_IN` и `PIPE_OUT` — управляющий FTP-поток;
- `DATA_BUF_BYTE`, `DATA_FLUSH`, `DATA_COPY_BLOCK` — пассивный поток данных;
- `BLOCK_READ_BYTE`, `BLOCK_READ_BUF`, `BLOCK_BUF_BYTE`, `BLOCK_WRITE_BYTE` — чтение и запись блока образа.

`devices.xml` подключает `SimplePipe` для управляющего канала, `BlockDevice` для файла `spo8_btrfs_block.img`, а также `SimplePic` и `SimpleClock`. Для FileZilla используется `devices_filezilla.xml`: C-адаптер прокидывает TCP control-порт 2121 и data-порт 2020 в два `SimplePipe` VM. Адаптер не хранит состояние FTP-сессии и не читает файловую систему; команды, текущий каталог и чтение файлов реализованы внутри VM-программы.

При старте `btrfs_mount` читает суперблок по смещению 0x10000, проверяет магическое значение `_BHRfS_M`, извлекает параметры корня и готовит таблицы для поиска. Для переходов по каталогам используется `fs_resolve_arg_path`, поддерживающий абсолютные и относительные пути, `.` и `...`.

`LIST` и `NLIST` обходят записи `DIR_ITEM` текущего каталога. `RETR` находит inode файла, получает размер из `INODE_ITEM`, физическое смещение extent из `EXTENT_DATA` и отправляет байты в пассивный FTP-поток. `COPY` для файла работает как `RETR`, а для директории рекурсивно проходит очередь подкаталогов. Передача данных выполняется блоками до 512 байт через `BLOCK_READ_BUF` и `DATA_COPY_BLOCK`, без побайтного чтения образа.

5 Результаты

Проверенные команды:

```
make -C SP08 local-demo
make -C SP08 local-unsupported
make -C SP08 remote-demo
make -C SP08 remote-unsupported
make -C SP08 remote-ftp-smoke
make -C SP08 remote-ftp-report-smoke
```

Положительный сценарий проверяет `SYST`, `NOOP`, `HELP`, `PASV`, `TYPE I`, `PWD`, `LIST`, `CWD`, `RETR`, `COPY`, `SIZE`, ошибочные `CWD/RETR` и `QUIT`.

Окончание положительного сценария содержит:

```
150 recursive directory copy follows
COPY file small.txt
150 inode=292 size=64 extent=btrfs
226 transfer complete
226 copy complete
550 not found
550 not found
221 bye
STATS cmd=24 lookup=16 stream=3281 gw=469
OK
```

Негативный сценарий с неподдержанной файловой системой завершается так:

```
SP08 BTRFS FTP
500 unsupported filesystem
OK_NEG
```

Smoke-тест FTP-клиента подтвердил, что листинг корня содержит SP08, а RETR /SP08/demo/small.txt возвращает содержимое файла:

```
SP08 Btrfs FTP demo file.
Read through RemoteTasks BlockDevice.
```

6 Выводы

Реализована работа с Btrfs-образом на уровне VM-программы: проверка суперблока, разрешение путей, обход каталогов, чтение extent и передача данных через пассивный FTP. Проверки через локальную VM, RemoteTasks и обычный FTP-клиент показали, что данные читаются из настоящего Btrfs-образа через BlockDevice, а не из файлов host-системы напрямую. Негативный сценарий подтверждает корректную обработку неподдержанной файловой системы.